



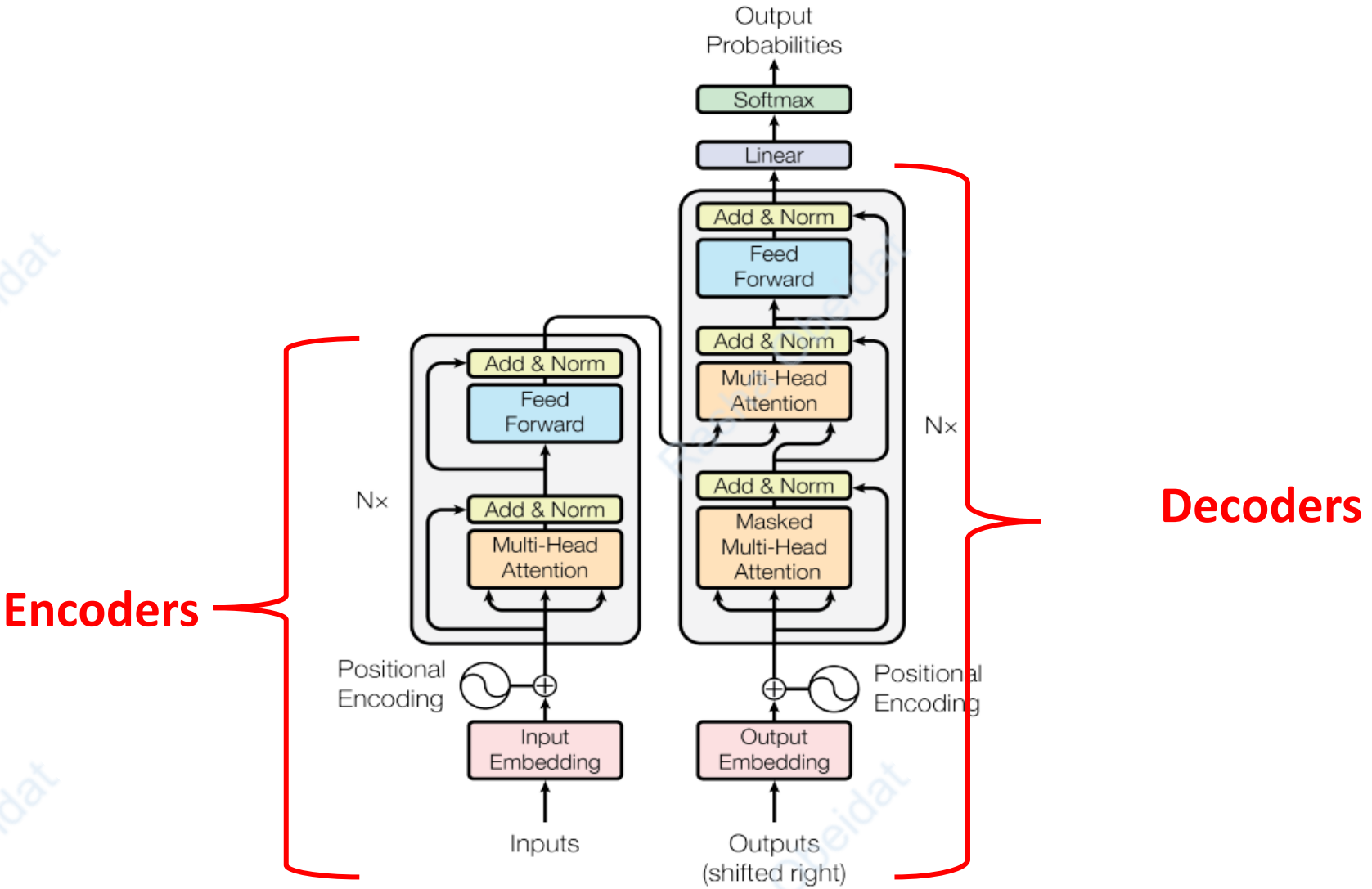
BERT

Rasha Obeidat, PhD

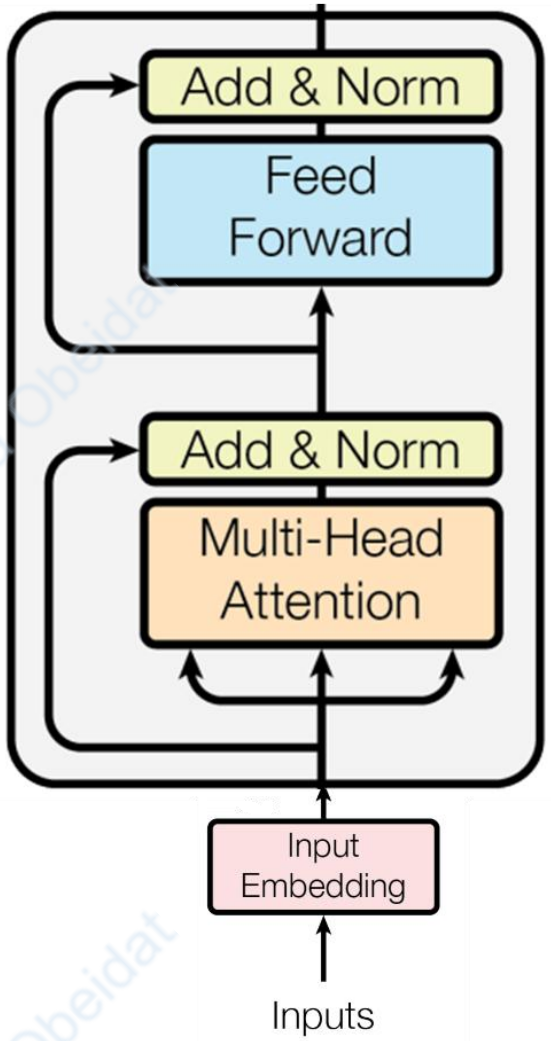
NLP

Rasha Obeidat

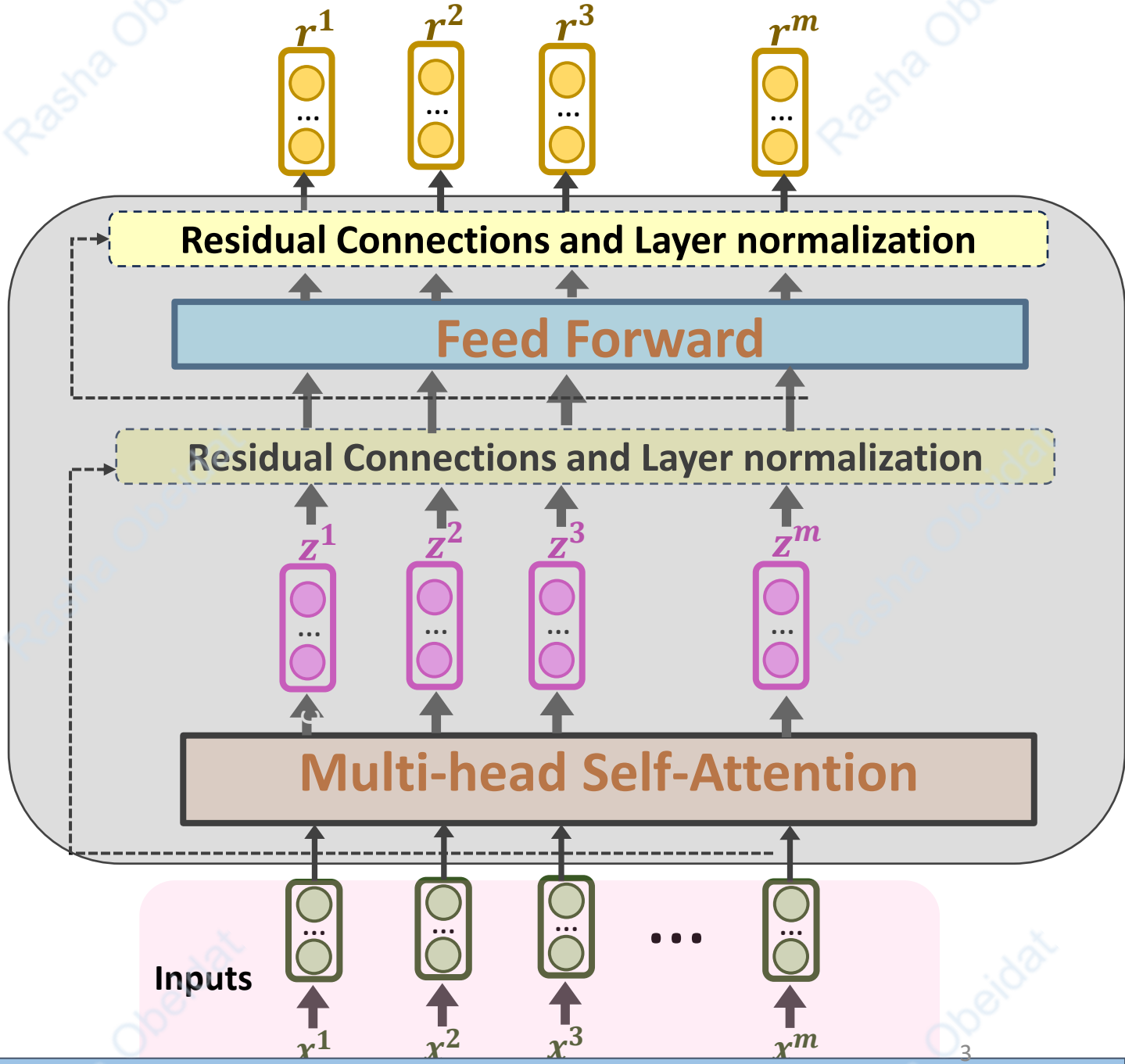
Transformer Encoder and Decoder



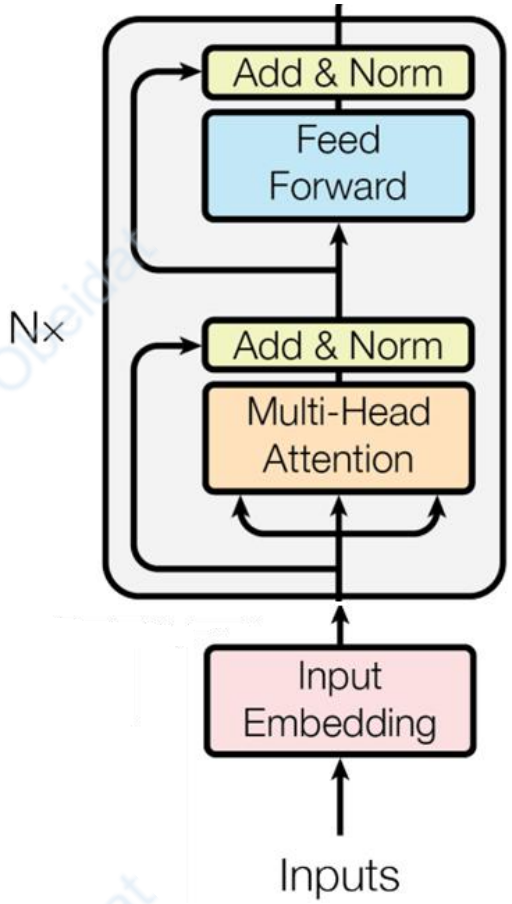
Transformer Encoder



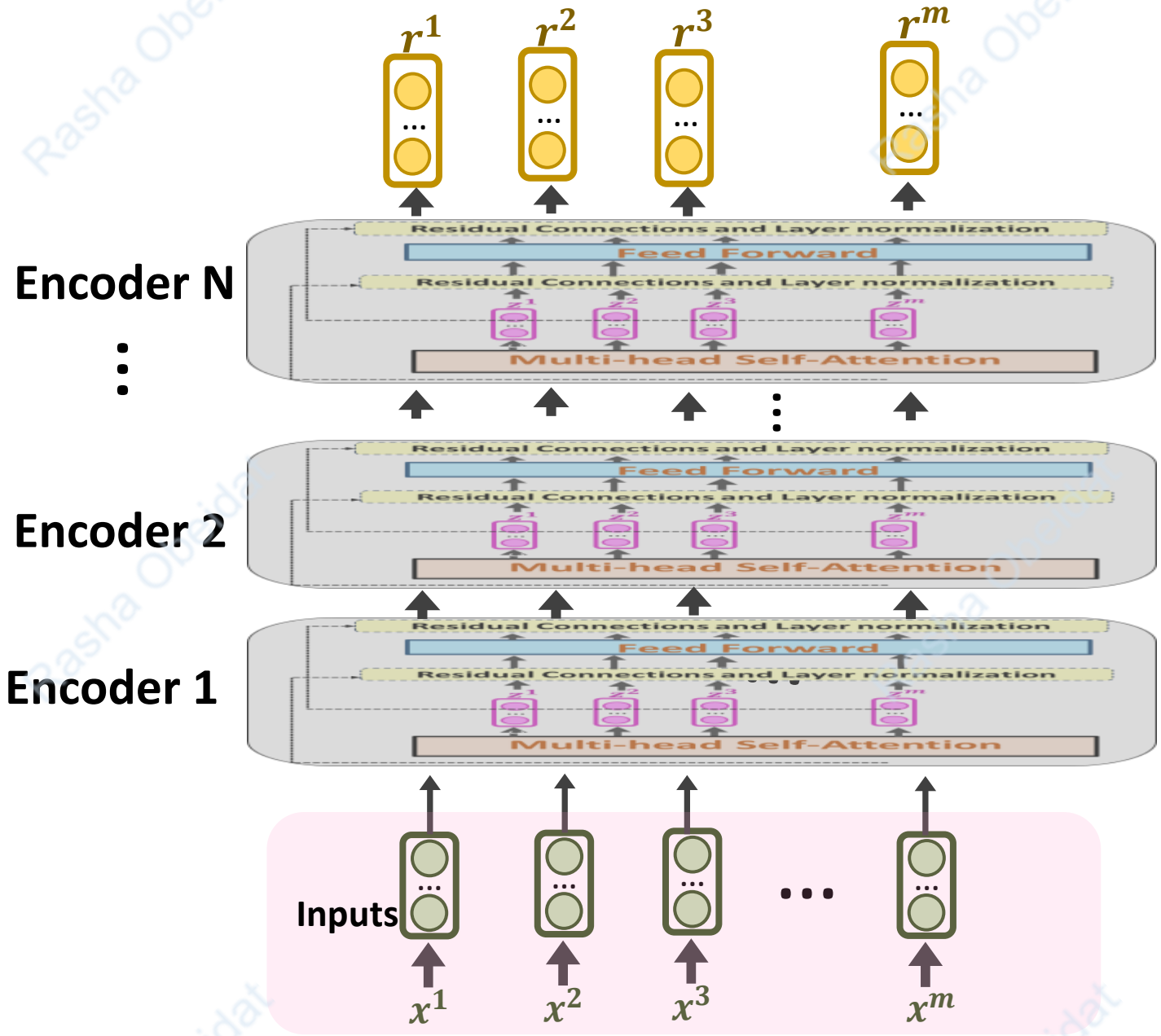
=



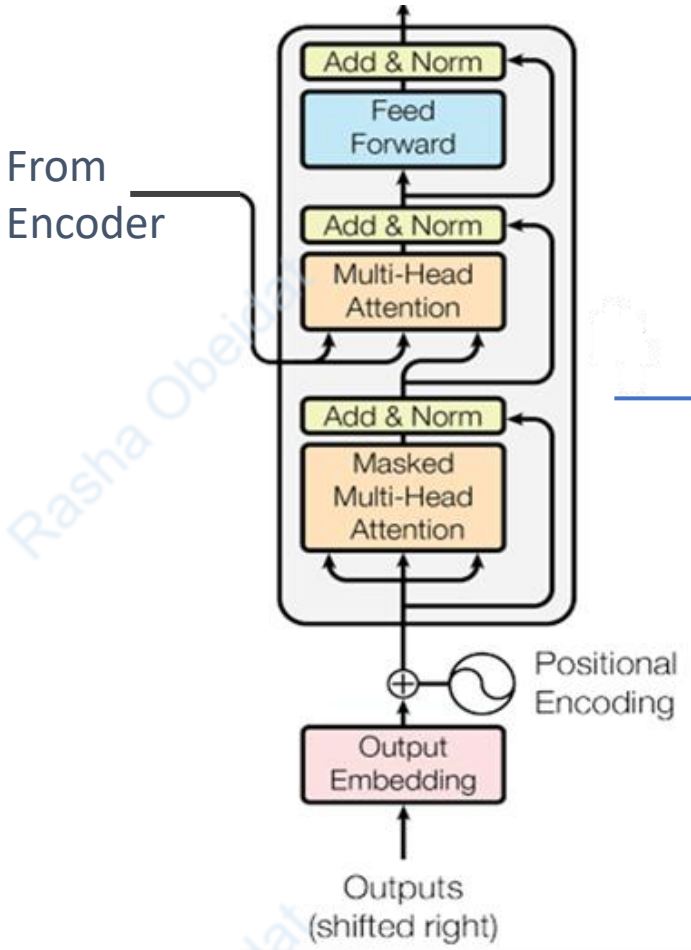
Stacking N encoders



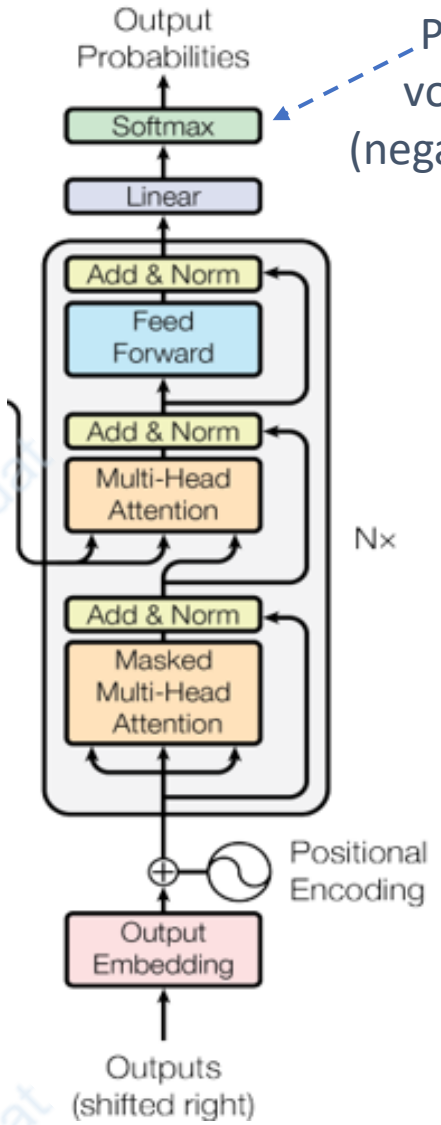
=



Transformer Decoder



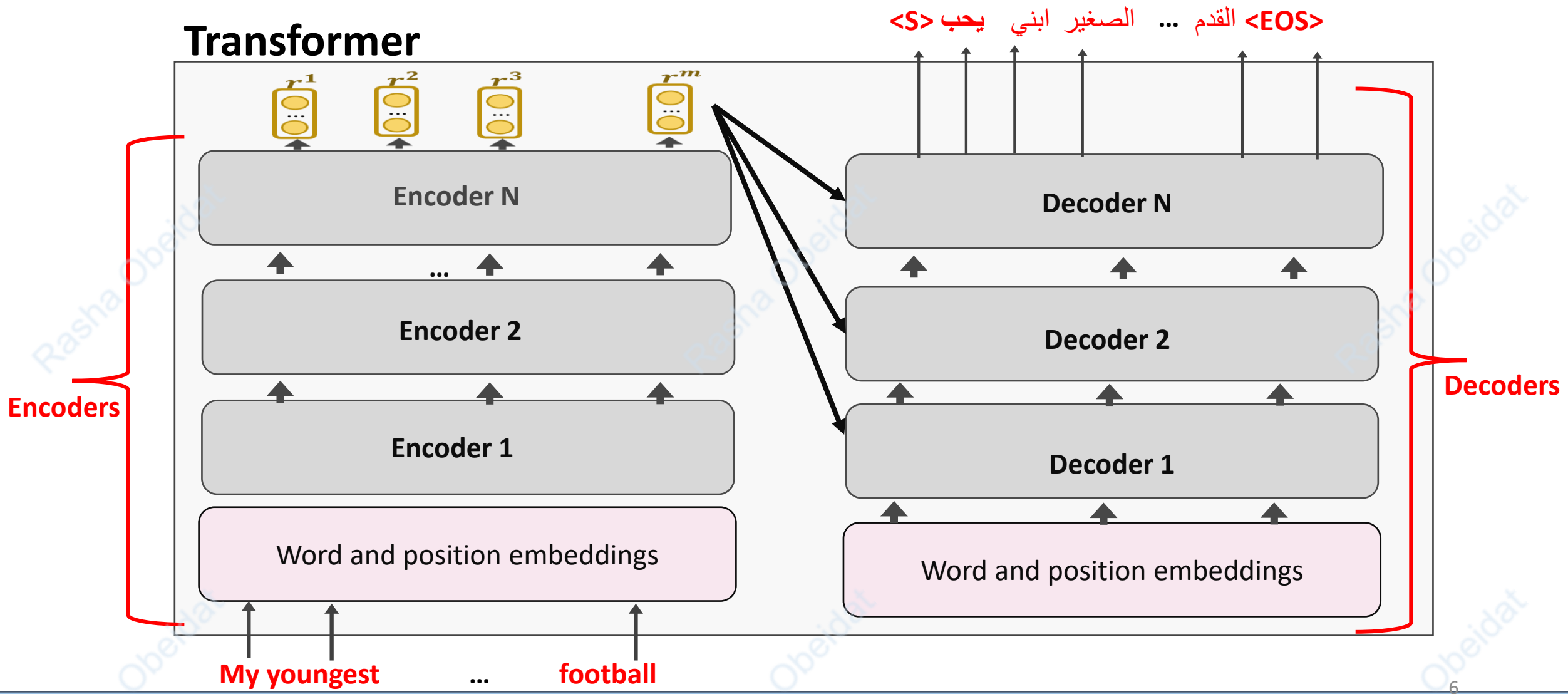
Stacking N decoders: each decoder receives input from decoder



Probability distribution over vocabulary of target language (negative sampling can be applied)

Transformer for MT

Loss Function: cross-entropy (predicting translated word)



From Transformer to BERT

- In the previous lectures, we explored the full Transformer model, composed of both ***encoders*** and ***decoders***, ideal for ***seq2seq*** tasks like MT.
- However, many NLP tasks, like sentence classification, Named Entity Recognition, or Machine Reading Comprehension, do not require generation, but deep ***understanding*** of input text.
- Today, we shift focus from **generation** to **understanding**.

From Transformer to BERT

- BERT addresses this need by:
  - Using only the encoder stack of the Transformer trained with bidirectional attention, seeing both left and right context.
  - Being pretrained on large corpora using masked language modeling and next sentence prediction.

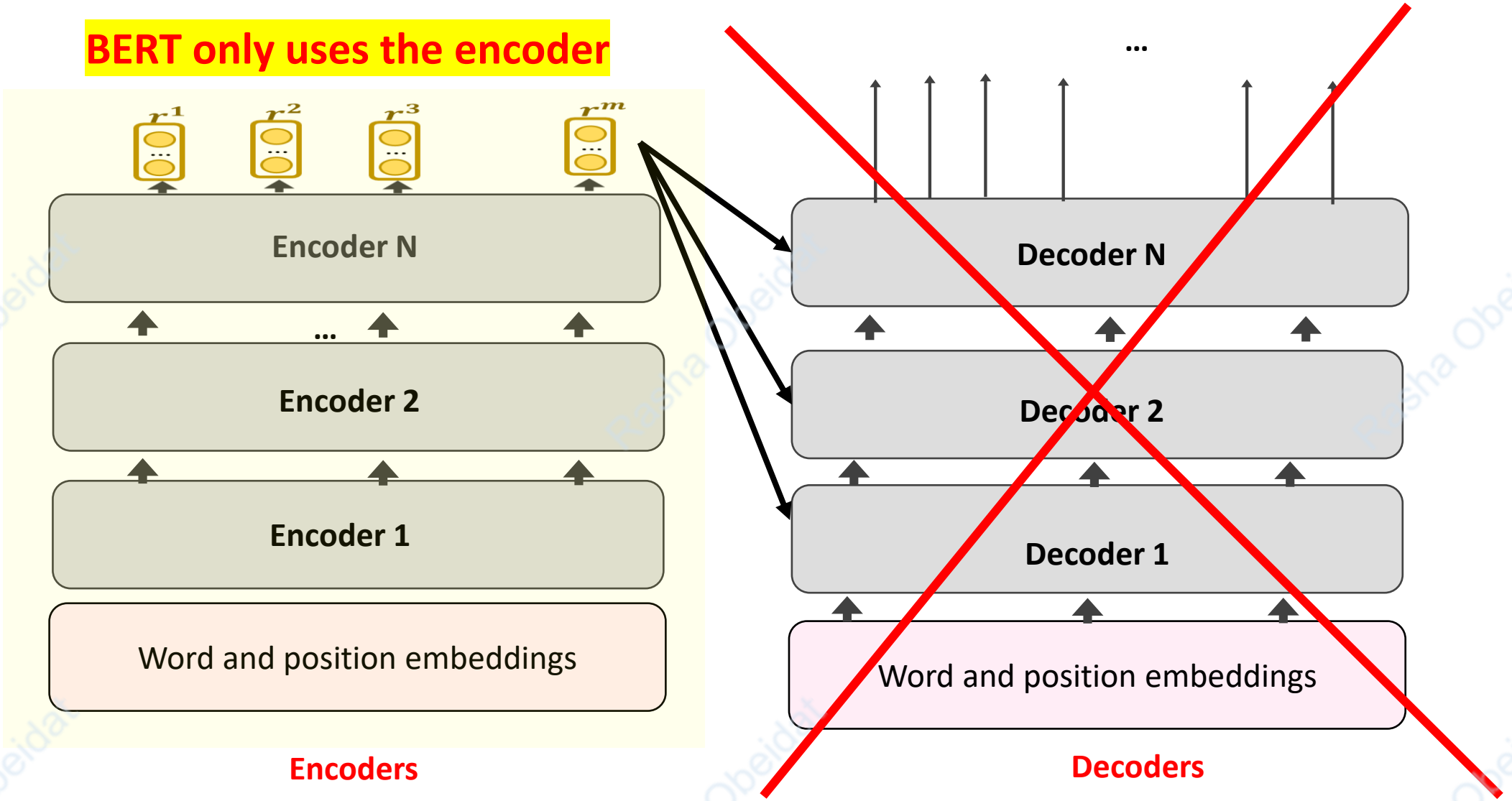


BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding

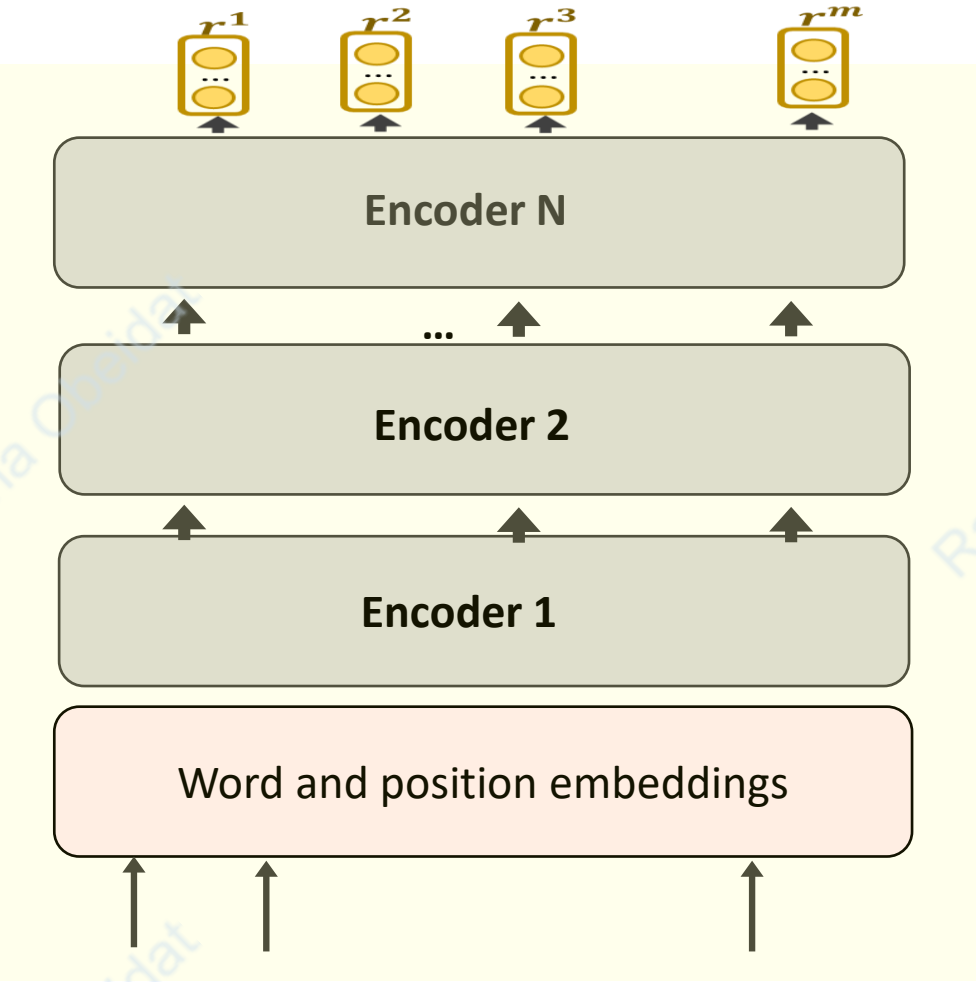
Jacob Devlin **Ming-Wei Chang** **Kenton Lee** **Kristina Toutanova**
Google AI Language
{jacobdevlin,mingweichang,kentonl,kristout}@google.com

Transformer

BERT only uses the encoder



BERT

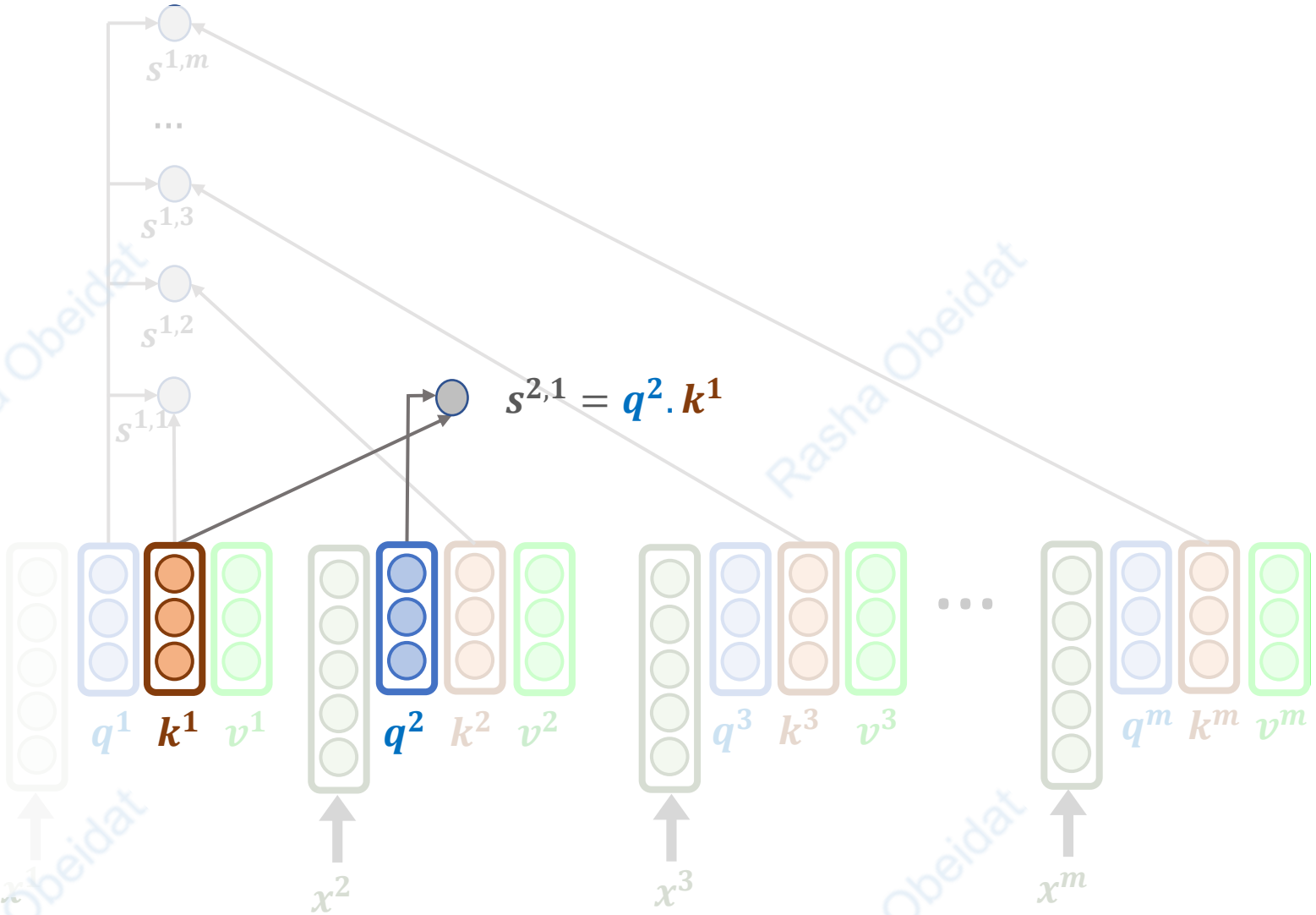


- **BERT : Bidirectional Encoder Representations from Transformers**
- **BERT** is a pre-trained model that uses only the **encoder** part of the Transformer architecture.
- **Bidirectional Context**: Unlike auto-regressive models, BERT processes text bidirectionally, considering both the left and right contexts simultaneously.
- Use **Bidirectional self-attention**!
- Called **Autoencoding Language Model.**

Bidirectional Self-attention

- **Is bidirectional self-attention a new concept**?
  - Answer: No!
  - **Unmasked** self-attention is naturally *bidirectional* — each token attends to all tokens in the input, left and right.
  - This means each token sees **the full sequence**, not just preceding tokens.
  - In the following slides, the word x^2 attended to the previous word x^1 (left context) and the next words $x^3$to x^m (right context).
  - Why? Full context → deeper understanding

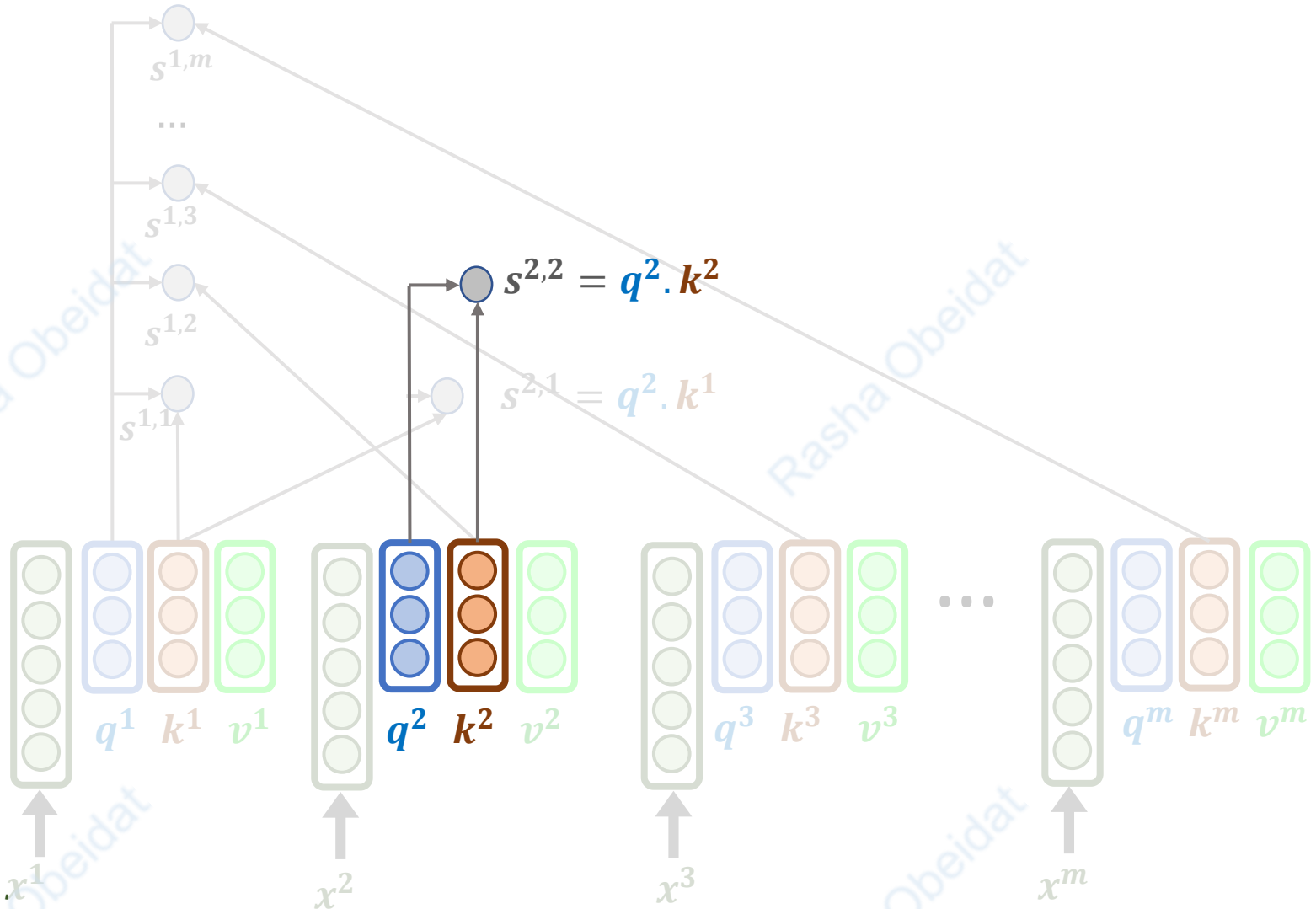
Bi-directional Self-Attention



Step3: Similarity Calculation (Dot Product)

- For a given **query** $q^i \; \forall \; i = 1, .., m$, the model computes its similarity score with all **keys** $k^j \; \forall \; j = 1, \dots, m$ in the sequence using a dot product.
- So, for each token x^i, we compute the similarity between its corresponding **query** and all tokens' **keys**.
- This results in a similarity score $s^{i,1}$, $s^{i,2}$, ..., $s^{i,m}$ for each $i = 1, 2, \dots, m$.
- This means that a similarity score is computed for each pair of tokens

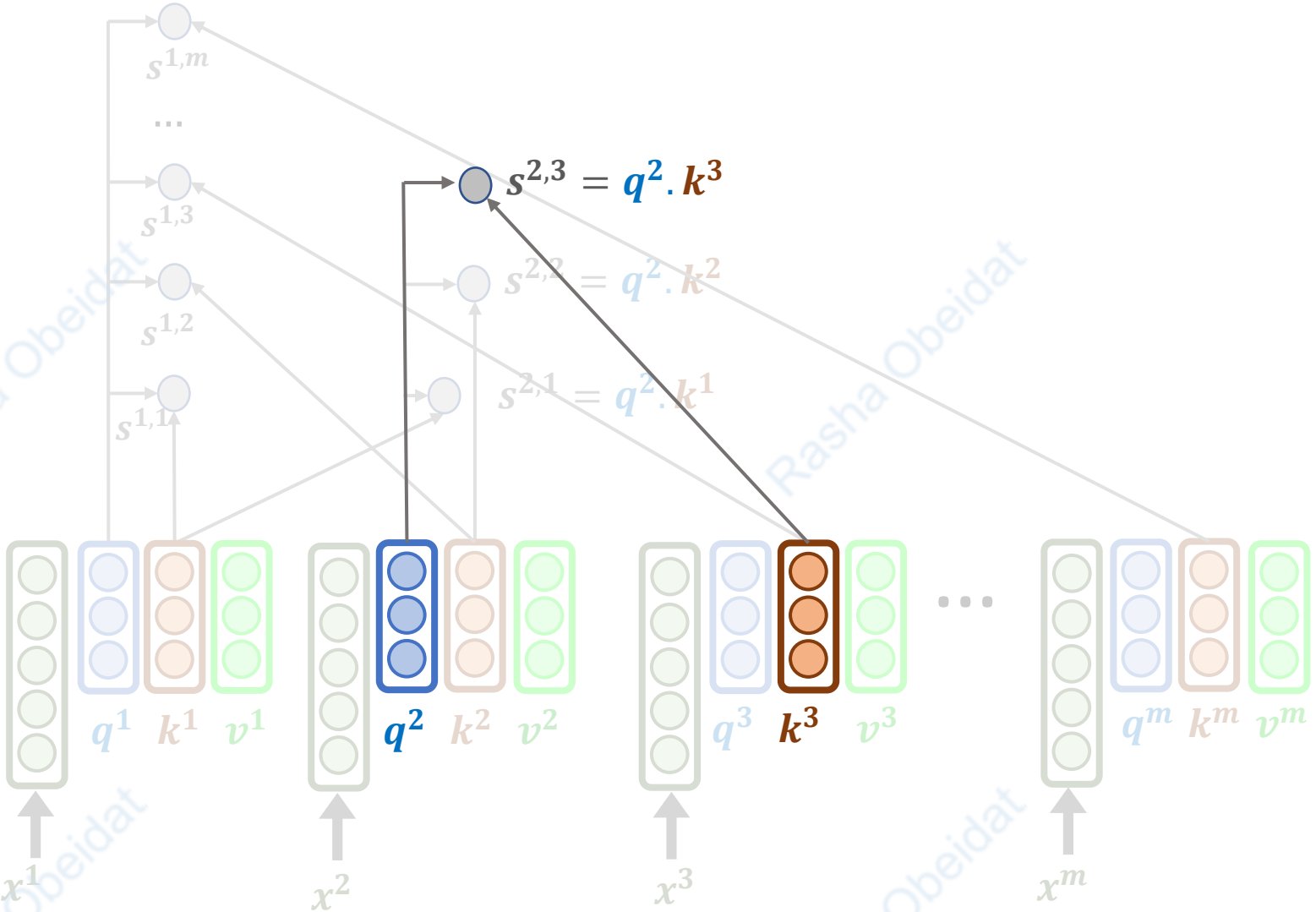
Bi-directional Self-Attention



Step3: Similarity Calculation (Dot Product)

- For a given **query** $q^i \forall i = 1, .., m$, the model computes its similarity score with all **keys** $k^j \forall j = 1, \dots, m$ in the sequence using a dot product.
- So, for each token x^i, we compute the similarity between its corresponding **query** and all tokens' **keys**.
- This results in a similarity score $s^{i,1}, s^{i,2}, \dots, s^{i,m}$ for each $i = 1, 2, \dots, m$.
- This means that a similarity score is computed for each pair of tokens

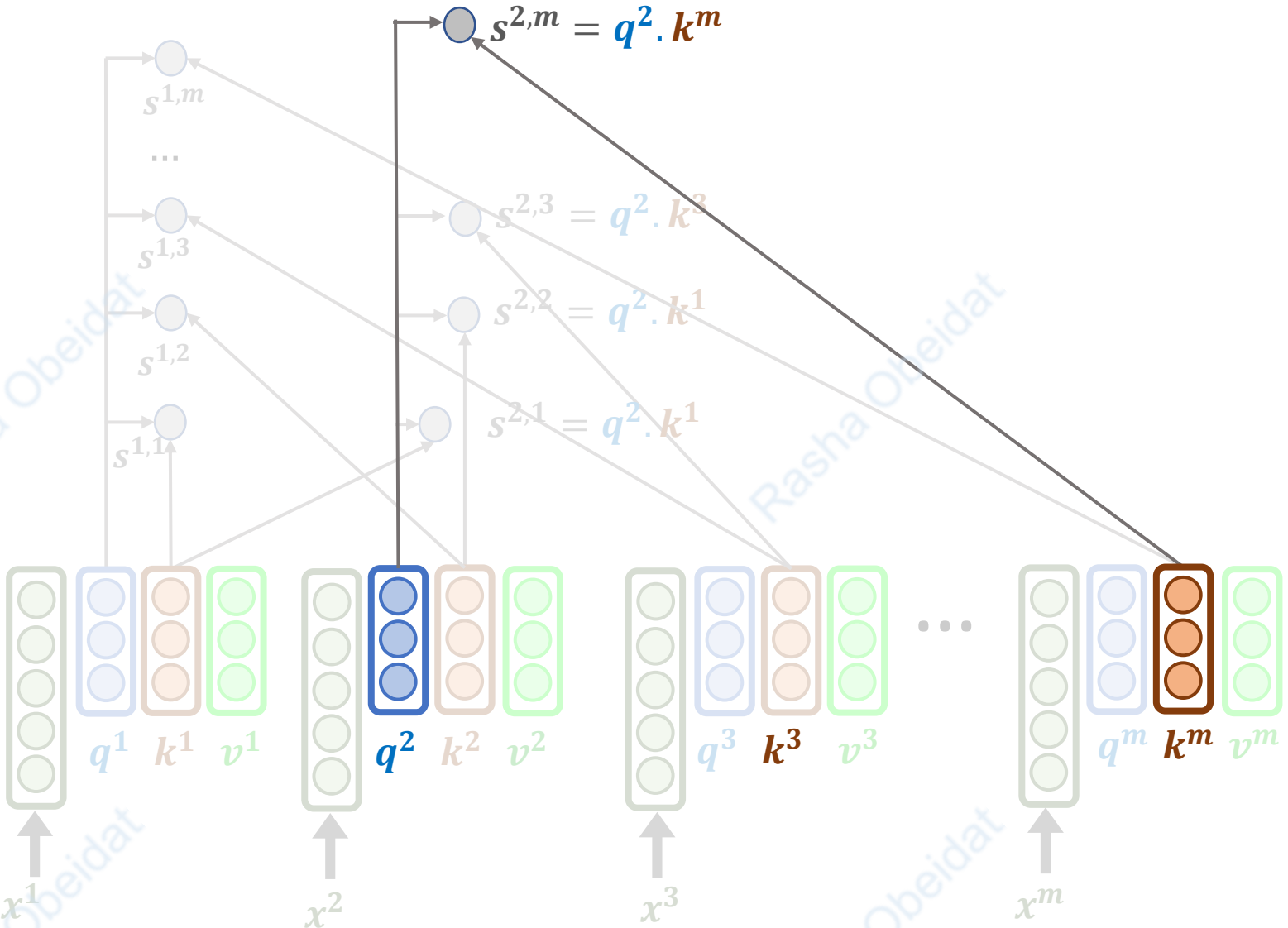
Bi-directional Self-Attention



Step3: Similarity Calculation (Dot Product)

- For a given **query** $q^i \; \forall \; i = 1, .., m$, the model computes its similarity score with all **keys** $k^j \; \forall \; j = 1, \dots, m$ in the sequence using a dot product.
- So, for each token x^i, we compute the similarity between its corresponding **query** and all tokens' **keys**.
- This results in a similarity score $s^{i,1}, s^{i,2}, \dots, s^{i,m}$ for each $i = 1, 2, \dots, m$.
- This means that a similarity score is computed for each pair of tokens

Bi-directional Self-Attention



Step3: Similarity Calculation (Dot Product)

- For a given **query** $q^i \forall i = 1, .., m$, the model computes its similarity score with all **keys** $k^j \forall j = 1, \dots, m$ in the sequence using a dot product.
- So, for each token x^i, we compute the similarity between its corresponding **query** and all tokens' **keys**.
- This results in a similarity score $s^{i,1}, s^{i,2}, \dots, s^{i,m}$ for each $i = 1, 2, \dots, m$.
- This means that a similarity score is computed for each pair of tokens

Bi-directional Self-Attention

- Is bidirectional self-attention a new concept?
  - Answer: No!
  - **Unmasked** self-attention is naturally *bidirectional* — each token attends to all tokens in the input, left and right.
  - This means each token sees **the full sequence**, not just preceding tokens.
  - In the following slides, the word x^2 attended to the previous word x^1 (left context) and the next words $x^3$to x^m (right context).
  - Why? Full context → deeper understanding
- On the other hand, Masked self-attention (e.g., used in the decoder) is not bidirectional , instead , it is left-to-right (***autoregressive***)

BERT: The goal

BERT: Bidirectional Encoder Representations from Transformers

- BERT is a Language Model (LM) that builds rich representations (rich contextual embeddings).
- A **Language Model** is a probabilistic model that assign probability distribution over sequences of words that matches distribution of a language.
  - Assigns a probability to a sequence of words and predicts the likelihood of the next word given the previous context.
  - It learns patterns in text and can:
    - **Predict** the next word in a sentence (e.g., in autocomplete)
    - **Generate** fluent text
    - **Understand** context for tasks like classification, translation, or summarization

Training Objectives of BERT

- **Key training objectives**:

  1. Masked Language Modeling (MLM):

  Predicts randomly masked words in a sentence using bidirectional context,

  2. Next Sentence Prediction (NSP):

  Determines whether two input sentences are sequentially related. I.e., Predict the if the second sentence of input truly follows the first sentence or not.

Input Representation in MLM

1. **Subword embeddings:** Input text is preprocessed and tokenized using **subword (WordPiece)** tokenization, and each **subword** is represented by an embedding .
2. **Adding Special tokens:**
    1. [CLS] at the start (for classification tasks)
    2. [SEP] to separate sentences (or to mark sentence boundary even if only one sentence is present)

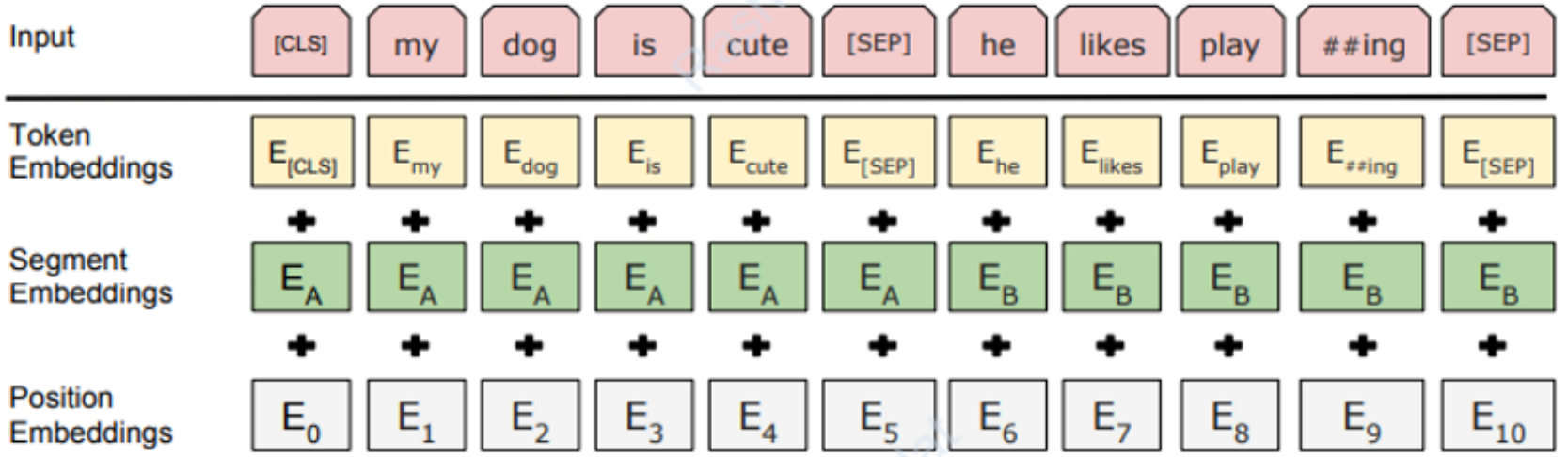
[CLS] I only read three **[mask]** this year. [SEP]

[SEP] .لشراء بعض الحاجات **[mask]** ذهبت الى [CLS]

Input Representation in MLM

3. Input embeddings

- **Token Embeddings** (as in Transformer)
- **Segment Embeddings** (Needed when two sentences are input together, more details are presented later).
- **Positional Embeddings** (as in Transformer)



BERT Paper: https://aclanthology.org/N19-1423.pdf

BERT's First Objective : Masked Language Modeling

- **Masked Language Modeling (MLM)** is a pretraining objective where BERT learns to predict randomly masked words in a sentence, using **both left and right context**.
- Examples:

1) Input: I only read three **[mask]** this year. → **Target**: books

2) Input: .لشراء بعض الحاجات **[mask]** ذهبت الى → **Target**: السوق

- The masked word can appear **anywhere** in the sentence to encourage the model to use full bidirectional context.

Masking in MLM

- **15%** of the input tokens in text sequence (e.g., sentence) are selected randomly for prediction (masking).
  - **80%** replaced with [MASK]

6 5 4 3 2 1

ذهبت الى **السوق** لشراء بعض الحاجات. → ذهبت الى **[MASK]** لشراء بعض الحاجات.

  - **10%** deliberately corrupted as wrong words by being replaced with a random word from the vocabulary

ذهبت الى السوق **لشراء** بعض الحاجات. → ذهبت الى السوق **شجرة** بعض الحاجات.

  - **10%** kept unchanged (but still predicted)

ذهبت الى السوق لشراء **بعض** الحاجات. → ذهبت الى السوق لشراء **بعض** الحاجات.

BERT is trained to predict the " السوق " at that position 3 , "لشراء" at position 4 and "بعض" at position 5

Masking in MLM

How many words will be masked in each text sequence?

Answer: Depending on the length of the sentence:

For example,

- If your sentence has **20 tokens**, about **3 tokens** will be selected for prediction and will be masked according to the 80-10-10 masking strategy.
- If your sentence has **100 tokens**, about **15 tokens** will be selected.

Example

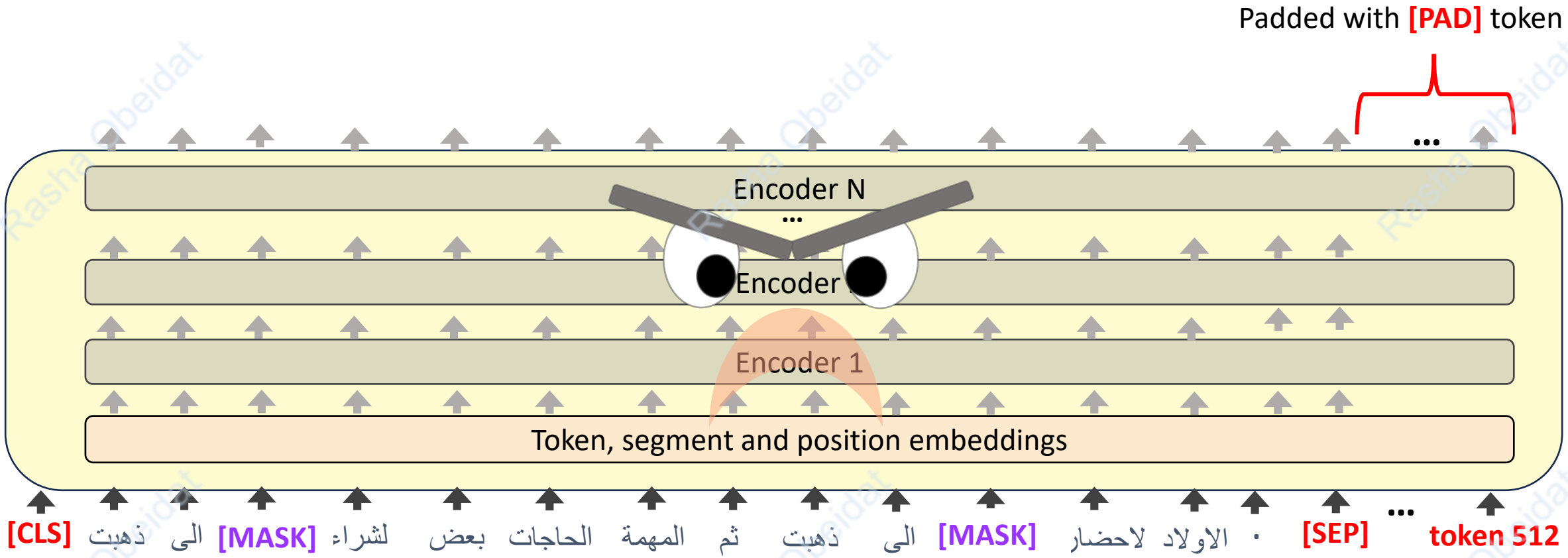
ذهبت الى السوق لشراء بعض الحاجات المهمة ثم ذهبت الى المدرسة لاحضار الاولاد.

•13 tokens total.
•15% of 16 ≈ 1.95 → round it → about 2 tokens selected randomly.
•Maybe " المدرسة " and " السوق " get selected.

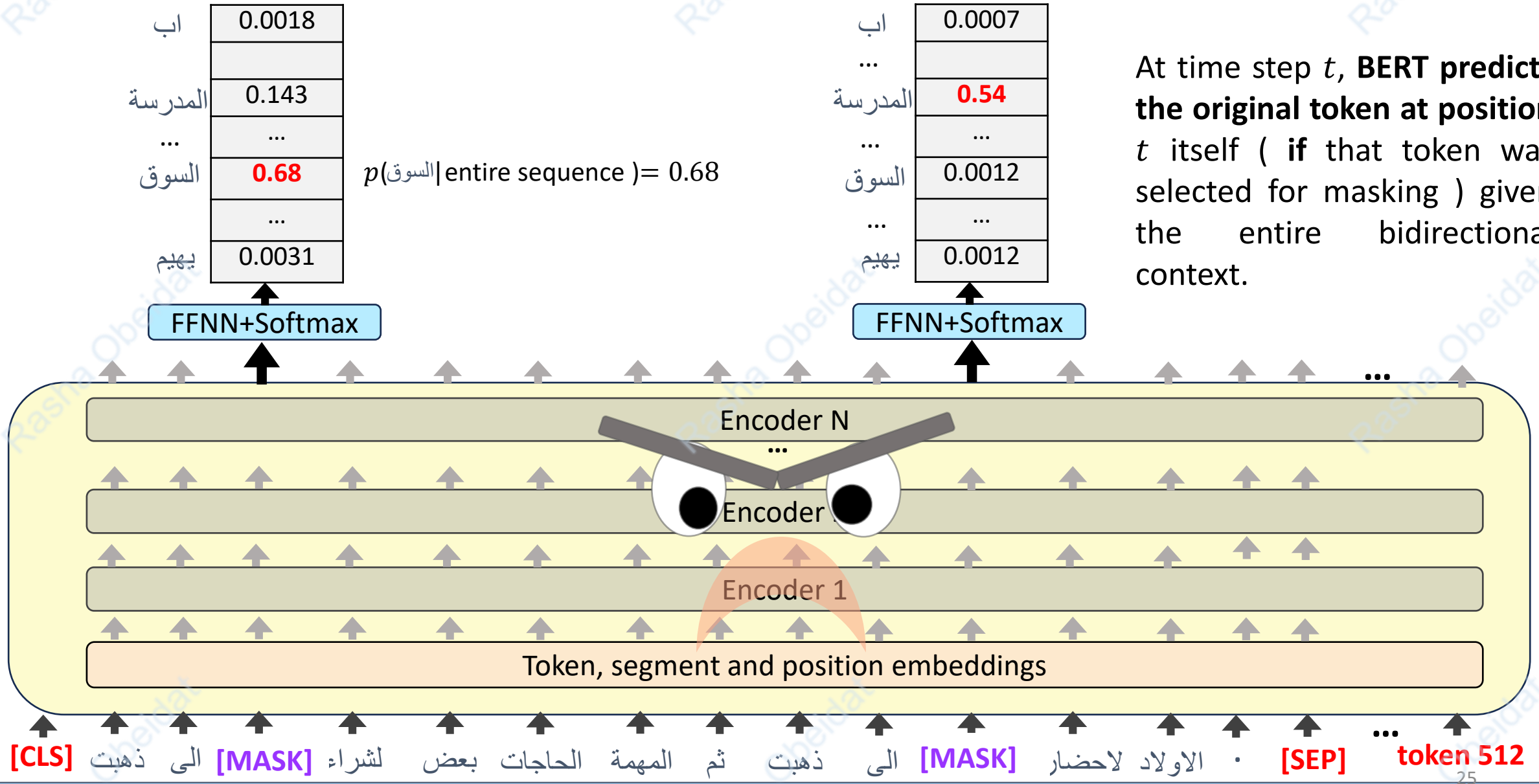
ذهبت الى **[MASK]** لشراء بعض الحاجات المهمة ثم ذهبت الى **[MASK]** لاحضار الاولاد.

Pre-Training Objective of BERT: MLM

Maximum sequence length in original BERT (base and large) is **512** tokens (after subword tokenization. Longer sequences are truncated, and shorter sequences are padded.



Pre-Training Objective of BERT: MLM



At time step t, **BERT predicts the original token at position** t itself (**if** that token was selected for masking) given the entire bidirectional context.

Training BERT with MLM objective

1. Forward Pass

- BERT produces a **hidden vector** for **every token**, but **only selected tokens** (masked, random, unchanged) are used for prediction.

2. Compute Prediction Loss

- For each selected token position: Predict the original word using a Softmax over the vocabulary
- Compute cross-entropy loss between the predicted distribution and the true label.

3. Aggregate

- Average the cross-entropy losses over all selected tokens in the input.

$$L(\theta) = -\frac{1}{|M|}\sum_{i \in M} \log(x^i | X_{masked})$$

- $\boldsymbol{L(\theta)}$: Total loss for one training example (averaged over masked tokens)
- $\boldsymbol{\theta}$: set of learnable parameters in BERT
- $\boldsymbol{x^i}$**:** The original token (the correct answer) at position i
- $\boldsymbol{x^{masked}}$**:** The full input sequence after masking (i.e., input after replacing some tokens with [MASK], random tokens, or leaving unchanged)

4. **Backward pass in BERT training for MLM:**

- **Compute the gradients** of the loss with respect to all the parameters (weights) in BERT using **backpropagation**.
- **Update** the parameters (using an optimizer like Adam) based on these gradients to minimize the loss

Masking in MLM

Why this complex masking?

- Prevents it from ***overfitting*** to the [MASK] token (which does not appear during fine-tuning).

  If BERT always saw [MASK] during pretraining, but never sees [MASK] during downstream fine-tuning (e.g., QA, classification), it would be overfit to recognizing [MASK] as a special clue.

- Trains a **robustness** model that is able handle **noisy or missing** information.
  - Replacing a token with a **random token** (10%) teaches BERT that Sometimes the input is noisy and that it cannot trust every word fully.
  - It learns to rely more **on context** to correct mistakes.
- Improves **generalization** during downstream tasks.

  Keeping the token unchanged teaches BERT that even when input looks perfect, it still have to guess which word was intended for prediction, thus, making BERT less dependent on a clear [MASK] signal and force it to **always predict based on context** → improves generalization.

Choosing Masking Percentages in MLM

How the 80%-10%-10% percentages were chosen?

- In the original BERT paper, these numbers were empirically chosen through internal experiments.
- They tried balancing two things:
  - **Learnability**: Easy clues to predict missing words.
  - **Generalization**: Realistic sentences without obvious signals.
- 80%-10%-10% is a **trade-off** that worked well:
  - BERT learns quickly enough.
  - BERT generalizes well to unseen tasks.

Dr. Rasha Obeidat ®

BERT's Second Objective : Next Sentence Prediction

- **Next Sentence Prediction (NSP)** learns to predict whether two input sentences **follow each other** in the original text.
- Examples:

1) Input :

- Sentence A: ذهبت إلى المكتبة .
- Sentence B: اشتريت بعض الكتب الجديدة .

Class *:* IsNext

Translation to English

I went to the bookstore.

I bought some new books.

2) Input :

- Sentence A: استيقظت مبكرًا هذا الصباح .
- Sentence B: شاهدت فيلمًا جديدًا في المساء .

Class *:* NotNext

Translation to English

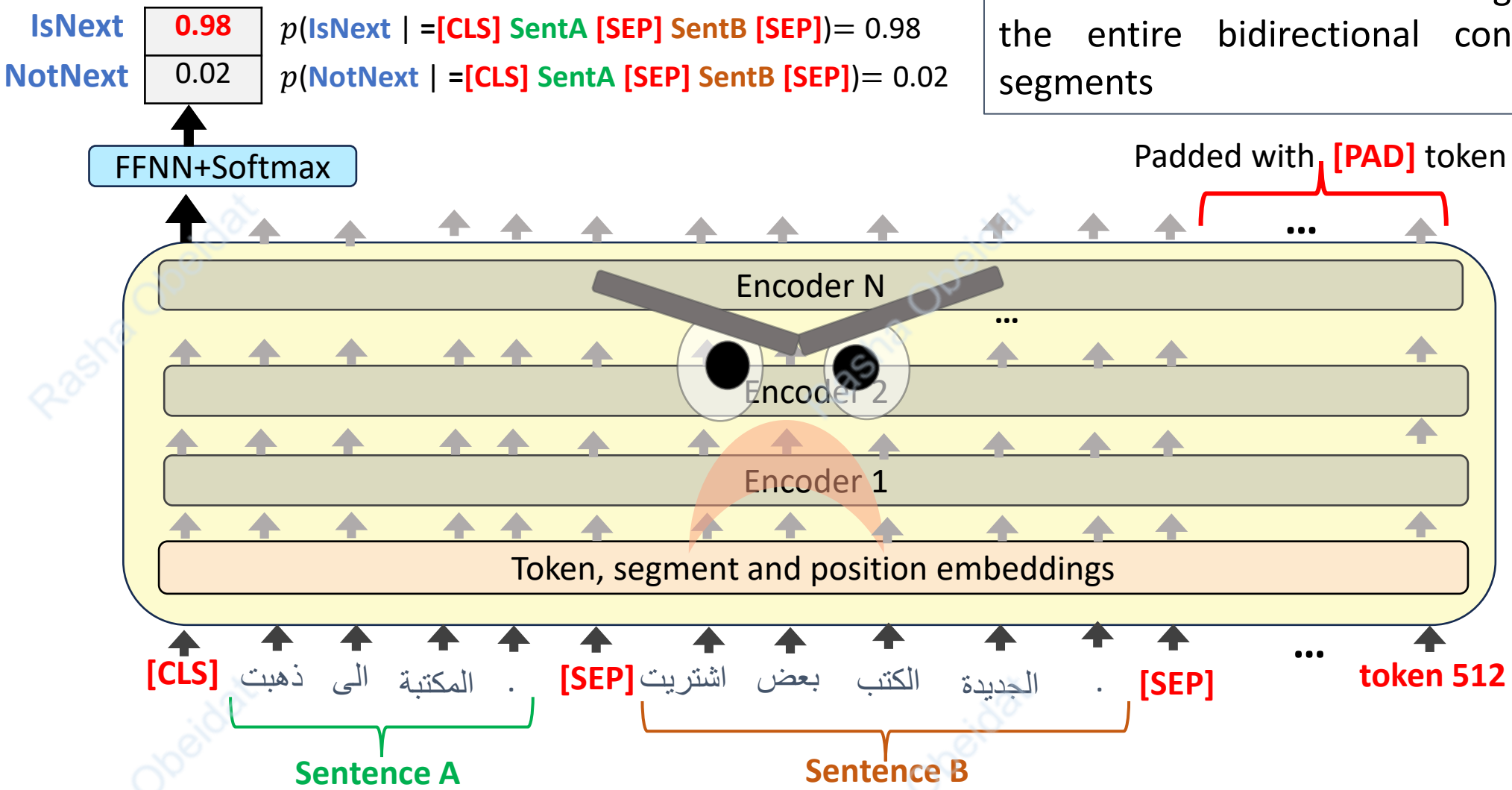
I woke up early this morning.

I watched a new movie in the evening.

- This helps BERT develop a deeper understanding of **relationships between sentences** — useful for tasks like Machine Reading Comprehension (MRC) and Natural Language Inference (NLI).
- The NSP task teaches BERT to model **longer document structure**, not just individual sentences.

Pre-Training Objective of BERT: NSP

BERT predicts whether the **second segment** follows the **first** one in the original text, given the entire bidirectional context of both segments



Preparing Training Data for NSP

- Two sentences are fed in at a time. Predict the if the second sentence of input truly follows the first one or not.
  - The first sentence is processed by [CLS]
  - Every two sentences are separated by a <SEP> token.
  - 50% of the time, the 2nd sentence is a randomly selected sentence from the corpus.
  - 50% of the time, it truly follows the first sentence in the corpus.
- To train BERT to not just *memorize* sequences, but to reason whether two sentences logically connect based on full meaning, not just word overlaps.

Rasha Obeidat

WordPiece (subword) tokenization

- **WordPiece** splits words into smaller subword units to reduce vocabulary size and handle rare or unknown words.
- **Example:**
  - **Word**: unhappiness
  - **WordPiece tokens**: un, ##happi, ##ness
  - The **##** prefix shows that a token is a ***continuation*** of a previous subword.

Advantages:

- Handles **out-of-vocabulary (OOV)** words.
- Captures **morphological features** (prefixes/suffixes).
- Encourages **shared embeddings** across similar words:
  - happy → happi
  - happiness → happi, ness

WordPiece (subword) tokenization

- **How WordPiece Learns Subwords:**
  - Starts with **all characters** as initial vocabulary.
  - Uses a **greedy algorithm** to merge the most frequent character pairs (or tokens).
  - Builds a vocabulary of the most frequent subword units.
  - Tokenization uses **longest-match-first (greedy)** approach
- For Example:
  - In English, the prefixes ***un*** , ***in*** , ***im*** , ***dis*** are captured since they are very frequent, and similarly for suffixes like ***tion***, ***sion***, **ied** etc.
  - For Arabic, WordPiece often learns:
    - Common prefixes like ال, بال,
    - Suffixes like ها, نا, ون, ات
- **Note:** Special tokens like [CLS], [SEP], [MASK] are **added after** WordPiece tokenization and are treated as full tokens.

The Separator [SEP]

- [SEP] is a special **separator token** used to indicate **boundaries** between segments in BERT.
- It **does not replace** natural punctuation (like periods . or question marks ؟) — it's added on top of regular text.
- **Why use [SEP]?**
  - Marks the **end of a sentence** or segment.
  - Enables **consistent input format** during pretraining.
  - Allows BERT to support tasks with **two segments** (e.g., NSP, NLI, QA).

The Separator [SEP]

- The use of **[SEP]** In pretraining
- In **MLM**

[CLS] sentence. [SEP]

- In **NSP**

[CLS] sentence A [SEP] sentence B [SEP]

- **In downstream tasks**

In NLI (Natural Language Inference) :

[CLS] premise [SEP] hypothesis [SEP]

In MRC (Machine Reading Comprehension) :

[CLS] passage [SEP] question [SEP]

More Examples on The Use Of [SEP]

Task	Input Format	Use of `[SEP]`
Text Classification (e.g., sentiment analysis)	`[CLS] text [SEP]`	One `[SEP]` at the end of the input text.
Named Entity Recognition (NER)	`[CLS] sentence [SEP]`	One `[SEP]` at the end. Per-token classification.
Question Answering (e.g., SQuAD)	`[CLS] question [SEP] context [SEP]`	Two `[SEP]` s separate the question and passage.
Natural Language Inference (NLI)	`[CLS] premise [SEP] hypothesis [SEP]`	Two `[SEP]` s separate the two sentence segments.
Paraphrase Detection / Similarity	`[CLS] sentence1 [SEP] sentence2 [SEP]`	Used just like NLI.
Sentence Pair Classification	`[CLS] sentence1 [SEP] sentence2 [SEP]`	Same structure — two segments.

Segment embeddings

Purpose of Segment Embeddings

Help BERT **distinguish** between different parts of the input when two text sequences (sentences) are fed together by telling the model which tokens belong to ***Sentence A***, and these tokens belong to ***Sentence B***

How Segment Embeddings Work

- Each token gets an **additional Segment ID**:
  - Tokens from the first sequence (Sentence A) are assigned **Segment A embedding (ID = 0).**
  - Tokens from the second sequence (Sentence B) are assigned **Segment B embedding (ID = 1).**
- The Segment Embedding is a learned dense vector with the same size as Token and Positional Embeddings (in original BERT , it is ***768***).
- All three embeddings are added ***elementwise*** to form the input vector.

Segment embeddings (MLM vs NSP)

- **In MLM**
  - All tokens are assigned Segment A (ID = 0).
  - Segment Embedding is still added but carries no distinguishing information.
- **In NSP**
  - Tokens from the first sentence (Segment A) are assigned the Segment A embedding (ID = 0).
  - Tokens from the second sentence (Segment B) are assigned the Segment B embedding (ID = 1).
  - Segment Embeddings help BERT distinguish which tokens belong to Sentence A and which belong to Sentence B.
- **In Downstream Tasks**
  - If there are more than two sentences, all tokens after the first [SEP] are assigned the Segment B embedding (ID = 1).
  - Advanced models (like ***Longformer***, ***BigBird***) extend BERT for longer or multi-segment inputs.

Dr. Rasha Obeidat ®

- These slides are the intellectual property of **Dr. Rasha Obeidat®**.
- The slides may **not be modified or redistributed** without explicit permission.
- **Contact:**
Obeidat.rm@gmail.com
Rmobeidat@just.edu.jo



Rasha Obeidat